

Scientific Computation

Federico Formica

Material by: Prof. Matthew Giamou

Nov. 20, 2025

Lecture 29 - Adaptive Integration & Ordinary Differential Eq. (ODEs)

Summary of the Adaptive Simpson's Rule

For inputs f , $[a, b]$, and ϵ_{tol} the adaptive Simpson's rule proceeds as follows:

1. $m = \frac{a+b}{2}$

2. Compute $S_1 = S(a, b)$ and $S_2 = S(a, m) + S(m, b)$

$\epsilon \quad \epsilon$

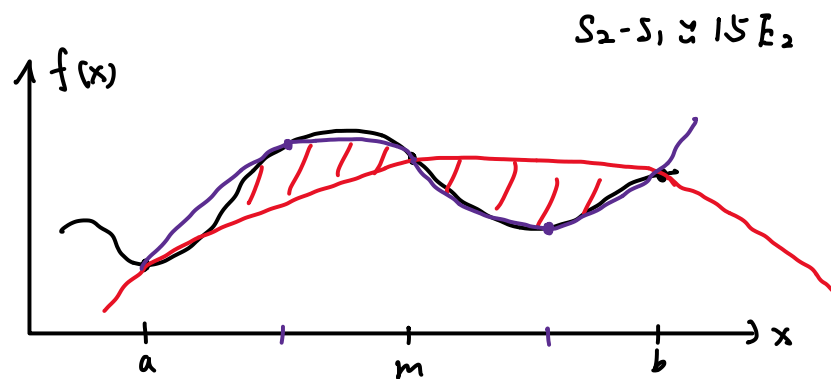
$$3. \bar{E}_2 = \frac{S_2 - S_1}{15}$$

4. If $|\bar{E}_2| \leq \epsilon_{tol}$, return $S_2, \bar{E}_2$.

Otherwise apply Adaptive Simpson's rule recursively on $[a, m]$ and $[m, b]$ with $\frac{\epsilon_{tol}}{2}$

8

Visualizing the Adaptive Simpson's Rule



This is a common strategy in scientific comp.

- Adaptive mesh refinement
- Multigrid methods for ODEs
- Octrees in vision and graphics

9

Why It Works

Let's make sure that our successive decrease of the error tolerance ($\epsilon_{\text{tol}} \rightarrow \epsilon_{\text{tol}}/2$ for each split) is mathematically sound:

$$I_f = \int_a^b f(x) dx, \quad I_1 = \int_a^m f(x) dx, \quad I_2 = \int_m^b f(x) dx$$

$\Rightarrow I_f = I_1 + I_2$, In each "branch" we have

$$|I_1 - Q_1| \leq \frac{\epsilon_{\text{tol}}}{2} \quad \text{and} \quad |I_2 - Q_2| \leq \frac{\epsilon_{\text{tol}}}{2}$$

↑
adaptive quadrature
for this branch

$$\begin{aligned} |I - Q| &= |(I_1 + I_2) - (Q_1 + Q_2)| \\ &= |(I_1 - Q_1) - (I_2 - Q_2)| \end{aligned}$$

$$(\text{tri. inequality}) \leq |I_1 - Q_1| + |I_2 - Q_2| \leq \epsilon_{\text{tol}}$$

A Word of Warning

In order to efficiently approximate the error term recursively, we assumed that $f^{(4)}$ did not fluctuate too wildly on $[a, b]$.

If this assumption is inaccurate, it may take too many iterations,

so set a maximum recursion depth, or be careful with setting

ϵ_{tol} too small.

Lecture Outline

- Adaptive Integration
- **Refresher on ODEs**
- Forward Euler's method
- Backward Euler's method

Differential Equations

A differential equation is an equation involving one or more unknown functions and their derivatives. For example, the heat equation in 3 dimensions is

$$\frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2},$$

where $u(x, y, z, t)$ is the unknown solution describing the temperature across space and time. This is a **partial differential equation (PDE)** because it involves

and time. This is a partial differential equation (PDE) because it involves derivatives of multiple independent variables (x , y , z , and t).

In this course, we still deal with ordinary DEs, which only consider functions with one independent var.

$$y = y' \triangleq \frac{dy}{dt}$$

13

Initial Value Problems

We will be studying initial value problems (IVPs), which ask us to solve an ODE from a given initial condition over a time period of interest:

$$y'(t) = f(t, y), \quad y(a) = c$$

$\hookrightarrow$ initial condition

We will approximate $y(t)$ at points t_i in $[a, b]$ using numerical methods.

14

A First Example

Example 1: Consider the linear differential equation

$$e^{4t}$$

$$y' = -y + t.$$

We are not going to solve ODEs analytically in this course, but let's *confirm* that $y(t) = t - 1 + \alpha e^{-t}$ is the solution.

$$y' = \frac{dy}{dt} = 1 - \alpha e^{-t}$$

$$\begin{aligned} y' = -y + t &= -(t - 1 + \alpha e^{-t}) + t \\ &= 1 - \alpha e^{-t} \end{aligned}$$

15

An IVP for this Example

Example 1 cont.: Now consider the initial condition $y(0) = 5$. What is the particular solution?

$$y(0) = -1 + \alpha = 5$$

$$\alpha = 6$$

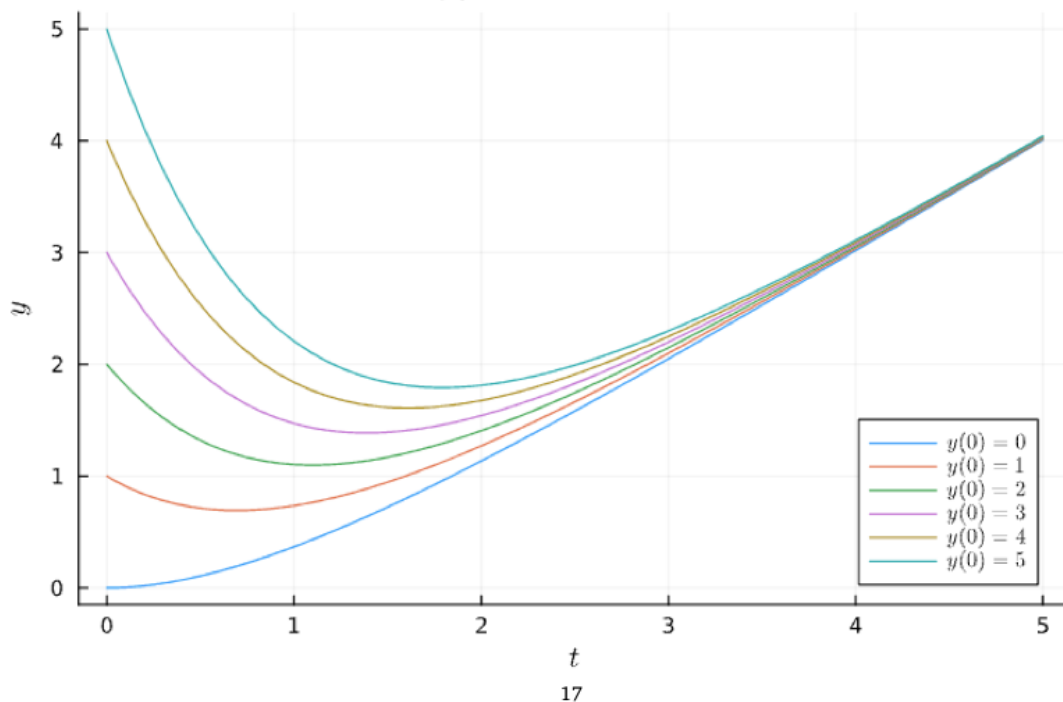
$$\Rightarrow y(t) = t - 1 + 6e^{-t}$$

is the solution to the IVP

Visualizing this IVP

Example 1 cont.: Here are solutions for different initial conditions $y(0) = c$:

$$y(t) = t - 1 + \alpha e^{-t}$$



General Form of ODEs

In general, we will study a system of n first-order equations in n variables:

$$y' \triangleq \frac{dy}{dt} = f(t, y), \quad f: \mathbb{R} \times \mathbb{R}^n \rightarrow \mathbb{R}^n.$$

↳ If f is linear, ODE is linear

↳ If f is non linear, ODE is non linear

If we have $y^{(n)} = f(t, y, y', \dots, y^{(n-1)})$, we can convert this n^{th} order ODE to 1st order.

$$y_1 = y, y_2 \triangleq y' = y_1', \dots, y_n = y^{(n)} = y_{n-1}'$$

To solve this, we need initial values for each $y_i(a)$, $i = 1 \dots n$

18

A Second (Order) Example

Example 2: The angular motion of a pendulum with a unit mass of 1 kg and a rigid massless rod can be described via the *second-order* nonlinear ODE

$$\frac{d^2\theta}{dt^2} = -g \sin \theta,$$

where $g \approx 9.81$ is the gravitational field strength on the Earth's surface.

Convert to first order:

$$y_1 = \theta, y_2 = \theta' = y_1' \Rightarrow \theta'' = y_2' = -g \sin \theta$$

$$\begin{bmatrix} y_1' \\ y_2' \end{bmatrix} = \begin{bmatrix} y_2 \\ -g \sin y_1 \end{bmatrix} \triangleq f(t, y)$$

Need $y_1(0)$ and $y_2(0)$ to specify an IVP

19

Lecture Outline

- Adaptive Integration
- Refresher on ODEs
- **Forward Euler's method**
- Backward Euler's method

Numerical Solutions

In general, nonlinear ODEs are **very** hard to solve exactly, and we must use numerical methods. We are considering IVPs of ODEs, so we have one independent variable. Consider first the scalar ($n = 1$) case:

$$y'(t) = f(t, y),$$

where we have an IVP with $y(t_0) = y_0$ and we want to solve for $y(t)$ on $[a, b]$.

Consider step size $h = \frac{b-a}{n}$, $n > 1$ is the number of steps

Let $t_0 = a$, $t_i = a + ih$, $i = 0, 1, \dots, n$ ($t_n = b$)

Euler's Method

Let's see what happens when we apply Taylor's theorem to $y(t)$ about each t_i and evaluate it at $t_{i+1} = t_i + h$:

$$y(t_{i+1}) = y(t_i) + y'(t_i) \underbrace{(t_{i+1} - t_i)}_h + \overbrace{\frac{1}{2} y''(\xi_i) (t_{i+1} - t_i)^2}^{\text{error}}$$

for some ξ_i between t_i and t_{i+1}

$$\approx y(t_i) + y'(t_i) \cdot h$$

$$= y(t_i) + h \cdot f(t_i, y(t_i))$$

Forward Euler Method Example

Example 3: Consider $y' = -y + t$ again, with $y(0) = y_0 = 5$ and compute the forward Euler's method with $h = 0.1$.

$$\begin{aligned} y_1 &= y_0 + h \cdot f(t_0, y_0) = y_0 + h(-y_0 + t_0) \\ &= 5 + 0.1(-5 + 0) = 4.5 = \frac{9}{2} \end{aligned}$$

$$\Rightarrow y_2 = y_1 + h(-y_1 + t_1) = 4.5 + 0.1(-4.5 + 0.1) \\ = 4.06$$

$$y_3 = 3.674$$

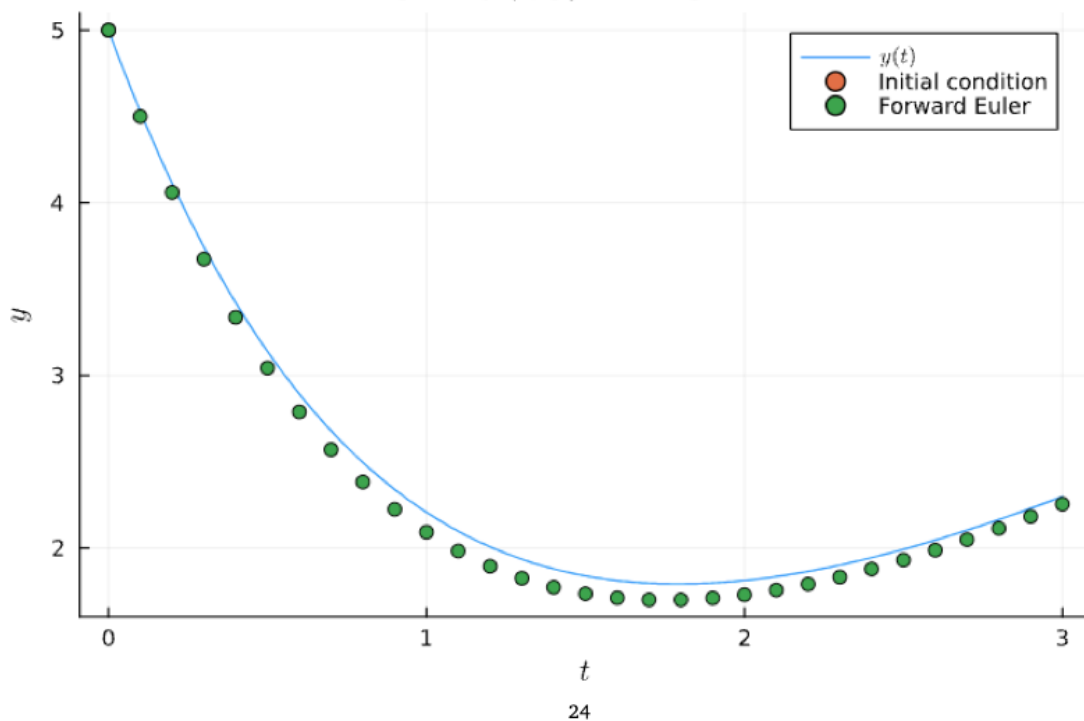
$\vdots$

23

Visualizing Forward Euler

Example 3 cont.: We can plot this solution in Julia:

$$y' = f(t, y) = -y + t$$



24

Lecture Outline

- Adaptive Integration
- Refresher on ODEs
- Forward Euler's method
- **Backward Euler's method**

Explicit vs. Implicit Methods

Forward Euler is called an *explicit method* because it takes the form

$$y(t_{i+1}) \approx y(t_i) + hf(t_i, y(t_i)), \quad (\text{explicit formula})$$

that is, the next value is computed *explicitly* as a function of the previous time step.

But we can also write the Taylor series about t_{i+1} and solve for $y(t_i)$:

$$y(t_i) = y(t_{i+1}) + y'(t_{i+1}) \underbrace{(t_i - t_{i+1})}_{-h} + \frac{1}{2} y''(\eta_i) \underbrace{(t_i - t_{i+1})^2}_{h^2}$$

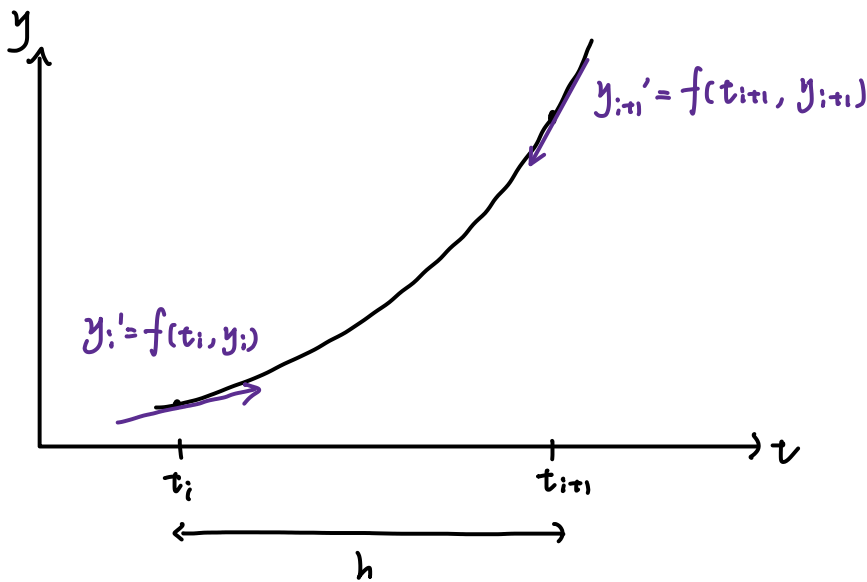
$$\approx y(t_{i+1}) - h \cdot f(t_{i+1}, y(t_{i+1}))$$

$$y(t_{i+1}) = y(t_i) + h \cdot f(t_{i+1}, y(t_{i+1}))$$

This is an implicit equation in $y(t_{i+1})$

Backward Euler

The result is called the **backwards Euler method** because we use the first derivative from the end of each subinterval (t_{i+1}) to solve for the start of each subinterval (t_i):



第一部分：自适应积分的算法与实现 (Adaptive Quadrature)

这部分从理论推导转向了具体的算法实现和工程注意事项。

1. 算法流程 (The Algorithm):

- 我们使用**递归 (Recursive)**的方式来实现。
- **核心步骤:**
 1. 计算粗糙解 S_1 (全区间) 和精细解 S_2 (分两半)。
 2. 利用公式 $\bar{E}_2 = \frac{S_2 - S_1}{15}$ 估算当前误差。
 3. **判断:** 如果误差 $|\bar{E}_2| \leq \epsilon_{tol}$, 则接受结果; 否则, 将区间一分为二, 对每一半递归调

用该算法。

- **关键细节：**在递归时，传递给子区间的容忍度必须**减半** ($\frac{\epsilon_{tol}}{2}$)，以保证总误差不超标。

2. 数学证明 (Why Tolerance/2?):

- 我们证明了为什么递归时容忍度要除以 2。
- 利用**三角不等式**，证明了总误差 $|I - Q|$ 小于等于左半边误差 $|I_1 - Q_1|$ 与右半边误差 $|I_2 - Q_2|$ 之和。
- 只要两边都小于 $\frac{\epsilon_{tol}}{2}$ ，总和就能保证小于 ϵ_{tol} 。

3. 直观理解与推广:

- 通过图像展示了 S_1 (红色平滑曲线) 和 S_2 (紫色贴合曲线) 的区别，它们之间的面积差就是误差来源。
- 这种“**只在需要的地方细分**”的思想是科学计算中的通用策略 (如自适应网格细化)。

4. 工程警告 (Warning):

- 算法依赖于“导数变化不大”的假设。如果函数极其震荡，假设失效，可能导致**无限递归**。
- **解决方案：**必须在代码中设置**最大递归深度 (Maximum Recursion Depth)** 作为“刹车”。

第二部分：常微分方程数值解 (Numerical ODEs)

这部分开启了一个全新的主题，介绍了如何用计算机解微分方程。

1. 定义与范围:

- 微分方程是包含未知函数及其导数的方程。
- 本课程只关注**常微分方程 (ODEs)**，即只有一个自变量 (通常是时间 t) 的情况。
- 首先学会了如何**验证**一个给定的函数是否是某个方程的解 (代入左边 = 代入右边)。

2. 核心技巧：降阶 (Reduction of Order):

- 虽然我们只学一阶方程 $y' = f(t, y)$ ，但这足以解决所有高阶问题。
- **方法：**任何 n 阶微分方程都可以通过引入新变量 (如 $y_2 = y'$, $y_3 = y''$ 等) 转化为一个包含 n 个方程的**一阶方程组**。
- **实例：**描述单摆运动的二阶方程 $\theta'' = -g \sin \theta$ ，被成功转换为了一个关于 θ 和 θ' 的一阶向量系统。

3. 数值求解基础:

- 由于解析解很难求，我们使用数值方法。
- **离散化 (Discretization)：**将连续的时间 t 切分为离散的网格点 $t_0, t_1, \dots, t_n$ ，步长为 h 。

4. 两种基本方法:

- 前向欧拉法 (Forward Euler) - 显式 (Explicit):
 - 利用当前点 t_i 的斜率向前走。
 - 公式: $y_{i+1} = y_i + h \cdot f(t_i, y_i)$ 。
 - 特点: 计算简单直接。
- 后向欧拉法 (Backward Euler) - 隐式 (Implicit):
 - 利用未来点 t_{i+1} 的斜率“指回”当前点。
 - 公式: $y_{i+1} = y_i + h \cdot f(t_{i+1}, y_{i+1})$ 。
 - 特点: 需要解方程 (未知数在两边), 计算难, 但稳定性好。

Backwards Euler's Method Example

Example 4: Consider $y' = -y + t$ once more, with $y(0) = y_0 = 5$ and compute the backward Euler's method with $h = 0.1$.

$$y_{i+1} = y_i + h \cdot f(t_{i+1}, y_{i+1}) = y_i + h(-y_{i+1} + t_{i+1})$$

↙
our approximation of
 $y(t_{i+1}) \rightarrow$ true value

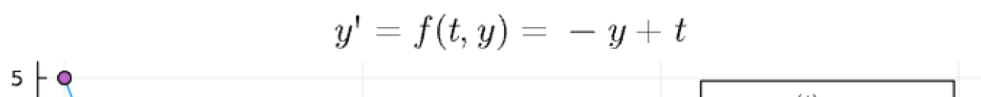
Gather y_{i+1} terms to the LHS!

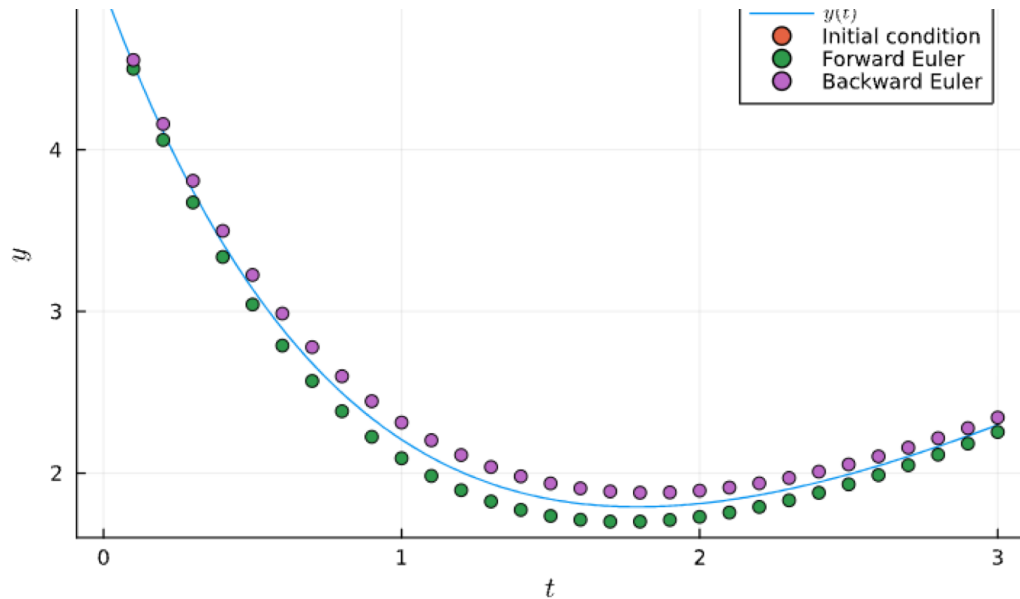
$$y_{i+1} + h \cdot y_{i+1} = y_i + h \cdot t_{i+1}$$

$$y_{i+1} = \frac{y_i + h \cdot t_{i+1}}{1+h}$$

Backwards Euler Visualized

Example 4 cont.: We can compare the backward and forward methods with Julia:





The General Case

In our example, we were lucky that the ODE was linear: it made solving the implicit equation in the backwards step quite simple. In general, we can use a root-finding method like Newton's method to solve

$$y(t_{i+1}) \approx y(t_i) + hf(t_{i+1}, y(t_{i+1}))$$

for $y(t_{i+1})$.

$$\text{Let } y \triangleq y(t_{i+1})$$

$$g(y) = y - y_i - h \cdot f(t_{i+1}, y) = 0$$

Newton's method:

$$y_{k+1} = y_k - \frac{g(y)}{g'(y)} \quad g'(y) = \frac{dg}{dy} = 1 - h \cdot \frac{\partial f}{\partial y}$$